==========

TAX INVOICE UPLOAD - DEPLOYMENT INSTRUCTIONS FOR LIVE SERVER

==========

ROLE-BASED ACCESS CONTROL:

==========

- ADMIN/ADMINISTRATOR: Can UPLOAD and DOWNLOAD tax invoices
- FINANCE: Can UPLOAD and DOWNLOAD tax invoices
- PAYEE: Can only DOWNLOAD tax invoices (NO upload access)
- Other roles: No access to this page

FILES TO DEPLOY:

==========

1. BCASApplication\Forms\DetailsForms\TaxInvoiceUpload.aspx
2. BCASApplication\Forms\DetailsForms\TaxInvoiceUpload.aspx.cs
3. Database_Scripts\Add_TaxInvoiceUpload_Page_Permissions.sql
4. Database_Scripts\QUICK_FIX_All_Roles.sql (for troubleshooting)
5. Database_Scripts\Complete_Diagnostic_PAYEE_Access.sql (for troubleshooting)
6. Database_Scripts\Final_Verification_Check.sql (for verification)

DATABASE CHANGES REQUIRED:

==========

A. CREATE STORED PROCEDURES:

---

1. STORED PROCEDURE: GetPaymentDetailsForTaxInvoice

   Action: CREATE

   -- This is a NEW stored procedure specifically for Tax Invoice Upload page

-- This procedure:

-- - Creates table CourseWisePaymentInvoiceStatus (if not exists)

-- - Syncs payment data with tracking table

-- - Returns payment details with invoice upload status (InvoiceUploadFile, Upload_Status)

IMPORTANT: The complete stored procedure SQL code is provided below in this document.
This is separate from the existing GetCourseWisePaymentDetails procedure used by
other pages, so it will not affect CourseWisePaymentDetails.aspx.

SQL Code:

```
USE [BCAS]
GO

SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO

CREATE PROCEDURE [dbo].[GetPaymentDetailsForTaxInvoice]
   @RoleName VARCHAR(50),
   @OrganisationCode INT = 0,
   @ScheduleID INT = 0,
   @FromDate DATE = NULL,
   @ToDate DATE = NULL
AS
BEGIN
   SET NOCOUNT ON;

   /* ---------------------------------------------------------------
     STEP 1 - create the tracking table the first time this proc ever runs.
```

```
   ------------------------------------------------------------------- */
    IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'CourseWisePaymentInvoiceStatus'
AND schema_id = SCHEMA_ID('dbo'))
    BEGIN
      CREATE TABLE dbo.CourseWisePaymentInvoiceStatus
      (
        Transaction_id          VARCHAR(50)    NOT NULL PRIMARY KEY,
        Course_Name             VARCHAR(255)   NULL,
        Schedule_ID             INT            NULL,
        Payee_OrgCode           INT            NULL,
        CourseSchedule          VARCHAR(400)   NULL,
        Reference_No            VARCHAR(100)   NULL,
        Amount_Recieved         DECIMAL(18, 2) NULL,
        Date_of_Payment         DATETIME       NULL,
        Payment_Status          VARCHAR(20)    NULL,
        ResponseDescription     VARCHAR(500)   NULL,
        Response_String         VARCHAR(MAX)   NULL,
        Settled_On              DATETIME       NULL,
        Settled_FileName        VARCHAR(255)   NULL,
        Payment_By              VARCHAR(100)   NULL,
        Payee_Organisation_Name VARCHAR(255)   NULL,
        Sponsoring_Organisation VARCHAR(255)   NULL,
        ProformaInvoiceNo       VARCHAR(100)   NULL,
        NoOfCandidates          INT            NULL,
        Fee_Amount              DECIMAL(18, 2) NULL,
        Service_Tax             DECIMAL(18, 2) NULL,
        Total_Fee_With_Service_Tax DECIMAL(18, 2) NULL,
        InvoiceUploadFile       VARCHAR(255)   NULL,
        Upload_Status           VARCHAR(20)    NOT NULL DEFAULT ('Not Uploaded'),
        LastSyncedOn            DATETIME       NOT NULL DEFAULT (GETDATE())
      );
```

```
END

/* ------------------------------------------------------------------
   STEP 2 - sync/upsert the tracking table from the live payment join.
   Deduplicates source data to prevent PK violations in MERGE.
   ------------------------------------------------------------------ */
;WITH LiveDataRaw AS
(
    SELECT
        cm.Course_Name, cs.Schedule_ID, pi.Payee_OrgCode,
        cm.Course_Name + ' - Exam Date: ' + CONVERT(VARCHAR(10), cs.Date_of_Exam, 103) AS
CourseSchedule,
        ot.Transaction_id, ot.Reference_No, ot.Amount_Recieved, ot.Response_Date AS
Date_of_Payment,
        CASE WHEN ot.Response_Status_Code = '0300' THEN 'DONE' ELSE 'NOT DONE' END AS
Payment_Status,
        ot.Response_Status_Message AS ResponseDescription, ot.Response_Parameters AS
Response_String,
        CAST(NULL AS DATETIME) AS Settled_On, CAST(NULL AS VARCHAR(255)) AS
Settled_FileName,
        ot.User_ID AS Payment_By, pom.Payee_Organisation_Name,
pom_sponsor.Organisation_Name AS Sponsoring_Organisation,
        pi.ProformaInvoiceNo, pi.NoOfCandidates, pi.Fee_Amount, pi.Service_Tax,
pi.Total_Fee_With_Service_Tax,
        ROW_NUMBER() OVER (
            PARTITION BY ot.Transaction_id
            ORDER BY
                CASE WHEN pom_sponsor.Organisation_Code IS NULL THEN 1 ELSE 0 END,
                pom_sponsor.Organisation_Code
        ) AS rn
    FROM Online_Transaction ot
    INNER JOIN Proforma_Invoice pi ON ot.ProformaInvoiceID = pi.ID AND
ot.Sponsoring_Organisation_Code = pi.Payee_OrgCode
    INNER JOIN Course_Schedule cs ON pi.Schedule_ID = cs.Schedule_ID
```

```
        INNER JOIN Course_Master cm ON cs.Course_Code = cm.Course_Code

        JOIN Payee_Organisation_Master pom ON pi.Payee_OrgCode =
pom.Payee_Organisation_Code AND ot.Sponsoring_Organisation_Code =
pom.Payee_Organisation_Code

        LEFT JOIN Payee_Sponsored_Organisation ps ON ot.Sponsoring_Organisation_Code =
ps.Payee_Organisation_Code AND ot.Sponsoring_Organisation_Code =
pom.Payee_Organisation_Code

        LEFT JOIN Sponsoring_Organisation_Master pom_sponsor ON
ps.Sponsoring_Organisation_Code = pom_sponsor.Organisation_Code

        WHERE ot.Response_Status_Code = '0300'

    ),

    LiveData AS

    (

        SELECT

            Course_Name, Schedule_ID, Payee_OrgCode, CourseSchedule, Transaction_id,
Reference_No,

            Amount_Recieved, Date_of_Payment, Payment_Status, ResponseDescription,
Response_String,

            Settled_On, Settled_FileName, Payment_By, Payee_Organisation_Name,
Sponsoring_Organisation,

            ProformaInvoiceNo, NoOfCandidates, Fee_Amount, Service_Tax,
Total_Fee_With_Service_Tax

        FROM LiveDataRaw

        WHERE rn = 1

    )

    MERGE dbo.CourseWisePaymentInvoiceStatus AS target

    USING LiveData AS src

        ON target.Transaction_id = src.Transaction_id

    WHEN MATCHED THEN

        UPDATE SET

            Course_Name             = src.Course_Name,

            Schedule_ID             = src.Schedule_ID,

            Payee_OrgCode           = src.Payee_OrgCode,

            CourseSchedule          = src.CourseSchedule,
```

```
        Reference_No            = src.Reference_No,
        Amount_Recieved         = src.Amount_Recieved,
        Date_of_Payment         = src.Date_of_Payment,
        Payment_Status          = src.Payment_Status,
        ResponseDescription     = src.ResponseDescription,
        Response_String         = src.Response_String,
        Payment_By              = src.Payment_By,
        Payee_Organisation_Name = src.Payee_Organisation_Name,
        Sponsoring_Organisation = src.Sponsoring_Organisation,
        ProformaInvoiceNo       = src.ProformaInvoiceNo,
        NoOfCandidates          = src.NoOfCandidates,
        Fee_Amount              = src.Fee_Amount,
        Service_Tax             = src.Service_Tax,
        Total_Fee_With_Service_Tax = src.Total_Fee_With_Service_Tax,
        LastSyncedOn            = GETDATE()
    WHEN NOT MATCHED BY TARGET THEN
      INSERT (Transaction_id, Course_Name, Schedule_ID, Payee_OrgCode, CourseSchedule,
Reference_No,
          Amount_Recieved, Date_of_Payment, Payment_Status, ResponseDescription,
Response_String,
          Payment_By, Payee_Organisation_Name, Sponsoring_Organisation, ProformaInvoiceNo,
          NoOfCandidates, Fee_Amount, Service_Tax, Total_Fee_With_Service_Tax,
          InvoiceUploadFile, Upload_Status, LastSyncedOn)
      VALUES (src.Transaction_id, src.Course_Name, src.Schedule_ID, src.Payee_OrgCode,
src.CourseSchedule, src.Reference_No,
          src.Amount_Recieved, src.Date_of_Payment, src.Payment_Status,
src.ResponseDescription, src.Response_String,
          src.Payment_By, src.Payee_Organisation_Name, src.Sponsoring_Organisation,
src.ProformaInvoiceNo,
          src.NoOfCandidates, src.Fee_Amount, src.Service_Tax, src.Total_Fee_With_Service_Tax,
          NULL, 'Not Uploaded', GETDATE());

    /* ---------------------------------------------------------------
```

```
    STEP 3 - return the up-to-date, role-scoped, filtered rows.
    ------------------------------------------------------------------ */
  SELECT
    Course_Name, Schedule_ID, Payee_OrgCode, CourseSchedule, Transaction_id, Reference_No,
    Amount_Recieved, Date_of_Payment, Payment_Status, ResponseDescription,
Response_String,
    Settled_On, Settled_FileName, Payment_By, Payee_Organisation_Name,
Sponsoring_Organisation,
    ProformaInvoiceNo, NoOfCandidates, Fee_Amount, Service_Tax,
Total_Fee_With_Service_Tax,
    InvoiceUploadFile, Upload_Status
  FROM dbo.CourseWisePaymentInvoiceStatus
  WHERE
    (@ScheduleID = 0 OR Schedule_ID = @ScheduleID)
    AND (@FromDate IS NULL OR CAST(Date_of_Payment AS DATE) >= @FromDate)
    AND (@ToDate IS NULL OR CAST(Date_of_Payment AS DATE) <= @ToDate)
    AND
    (
      UPPER(ISNULL(@RoleName, '')) LIKE '%ADMIN%'
      OR
      (
        UPPER(ISNULL(@RoleName, '')) LIKE '%PAYEE%'
        AND ISNULL(Payee_OrgCode, 0) = @OrganisationCode
      )
      OR
      (
        (
          UPPER(ISNULL(@RoleName, '')) LIKE '%SPON%'
          OR UPPER(ISNULL(@RoleName, '')) LIKE '%SPONSORING%'
        )
        AND ISNULL(Payee_OrgCode, 0) = @OrganisationCode
      )
```

```
    )
  ORDER BY
    Date_of_Payment DESC,
    Transaction_id DESC;
END
GO
```

---

2. STORED PROCEDURE: SaveSignedTaxInvoiceFile

Action: CREATE

SQL Code:

```
USE [BCAS]
GO

SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO

CREATE PROCEDURE [dbo].[SaveSignedTaxInvoiceFile]
        @pOrg INT,
        @scheduleId INT,
        @TransactionID VARCHAR(50),
        @FileName VARCHAR(500)
AS
BEGIN
        SET NOCOUNT ON;
```

```
        UPDATE dbo.CourseWisePaymentInvoiceStatus
        SET InvoiceUploadFile = @FileName,
              Upload_Status = 'Uploaded',
              LastSyncedOn = GETDATE()
        WHERE Transaction_id = @TransactionID
            AND Payee_OrgCode = @pOrg
            AND Schedule_ID = @scheduleId;
  END
  GO
```

B. ADD MENU ITEM TO MenuMaster TABLE:

SQL Code:

```
SET IDENTITY_INSERT [dbo].[MenuMaster] ON;

INSERT INTO [dbo].[MenuMaster]
(
        [MenuItemCode],
        [MenuDescription],
        [MenuLevel],
        [WhetherLeafMenu],
        [ParentMenuItem],
        [FormName],
        [Enter_By],
        [Enter_Date]
)
VALUES
```

```
(
	99,                                  -- Check if MenuItemCode 99 is available, if not use next
available number
	'Tax Invoice Upload',
	2,                                   -- MenuLevel (2 = sub-menu)
	'Y',                                 -- WhetherLeafMenu (Y = leaf/actual page)
	38,                                  -- ParentMenuItem (38 = Payments menu)
	'~/Forms/DetailsForms/TaxInvoiceUpload.aspx',
	'admin',
	GETDATE()
);

SET IDENTITY_INSERT [dbo].[MenuMaster] OFF;

VERIFICATION QUERY:
SELECT * FROM [dbo].[MenuMaster] WHERE MenuItemCode = 99;
```

C. ADD ROLE PERMISSIONS TO RoleForm TABLE:

IMPORTANT: This page supports THREE roles with DIFFERENT access levels:

- PAYEE: Download only (no upload)
- ADMIN/ADMINISTRATOR: Upload and download
- FINANCE: Upload and download

METHOD 1: Use the Automated Script (RECOMMENDED)

Run the complete script that handles all role variations:

File: Database_Scripts\Add_TaxInvoiceUpload_Page_Permissions.sql

This script will:

1. Add menu item to MenuMaster

2. Add permissions for all role variations (PAYEE, Payee, ADMINISTRATOR, Admin, FINANCE, Finance)

3. Refresh views and stored procedures

4. Show verification results

Simply execute this one file in SQL Server Management Studio.

METHOD 2: Manual SQL Commands (If script fails)

SQL Code:

```
-- Add PAYEE role (can download only)
IF NOT EXISTS (SELECT 1 FROM RoleForm WHERE RoleName = 'PAYEE' AND FormName = '~/Forms/DetailsForms/TaxInvoiceUpload.aspx')
BEGIN
    INSERT INTO [dbo].[RoleForm] ([RoleName], [FormName], [Enter_By], [Enter_Date])
    VALUES ('PAYEE', '~/Forms/DetailsForms/TaxInvoiceUpload.aspx', 'admin', GETDATE());
END

-- Add ADMINISTRATOR role (can upload and download)
IF NOT EXISTS (SELECT 1 FROM RoleForm WHERE RoleName = 'ADMINISTRATOR' AND FormName = '~/Forms/DetailsForms/TaxInvoiceUpload.aspx')
BEGIN
    INSERT INTO [dbo].[RoleForm] ([RoleName], [FormName], [Enter_By], [Enter_Date])
    VALUES ('ADMINISTRATOR', '~/Forms/DetailsForms/TaxInvoiceUpload.aspx', 'admin', GETDATE());
END

-- Add Admin role (alternate casing)
```

```
IF NOT EXISTS (SELECT 1 FROM RoleForm WHERE RoleName = 'Admin' AND FormName =
'~/Forms/DetailsForms/TaxInvoiceUpload.aspx')

BEGIN

   INSERT INTO [dbo].[RoleForm] ([RoleName], [FormName], [Enter_By], [Enter_Date])

   VALUES ('Admin', '~/Forms/DetailsForms/TaxInvoiceUpload.aspx', 'admin', GETDATE());

END


-- Add FINANCE role (can upload and download)

IF NOT EXISTS (SELECT 1 FROM RoleForm WHERE RoleName = 'FINANCE' AND FormName =
'~/Forms/DetailsForms/TaxInvoiceUpload.aspx')

BEGIN

   INSERT INTO [dbo].[RoleForm] ([RoleName], [FormName], [Enter_By], [Enter_Date])

   VALUES ('FINANCE', '~/Forms/DetailsForms/TaxInvoiceUpload.aspx', 'admin', GETDATE());

END


-- Add Finance role (alternate casing)

IF NOT EXISTS (SELECT 1 FROM RoleForm WHERE RoleName = 'Finance' AND FormName =
'~/Forms/DetailsForms/TaxInvoiceUpload.aspx')

BEGIN

   INSERT INTO [dbo].[RoleForm] ([RoleName], [FormName], [Enter_By], [Enter_Date])

   VALUES ('Finance', '~/Forms/DetailsForms/TaxInvoiceUpload.aspx', 'admin', GETDATE());

END


-- Refresh view and stored procedure

EXEC sp_refreshview 'vw_formname';

EXEC sp_recompile 'GetMenuData';



VERIFICATION QUERY:

-------------------

SELECT RoleName, FormName, Enter_Date

FROM [dbo].[RoleForm]
```

WHERE FormName = '~/Forms/DetailsForms/TaxInvoiceUpload.aspx'

ORDER BY RoleName;

-- You should see at least 3 roles (PAYEE, ADMIN/ADMINISTRATOR, FINANCE)

D. CREATE FOLDER STRUCTURE FOR FILE STORAGE:

Create the following folder structure on the server:

1. Go to: [WebRoot]\Download\
2. Create folder: TaxInvoice
3. Full path should be: [WebRoot]\Download\TaxInvoice\

NOTE: The application will automatically create subfolders for each

organization and schedule when invoices are uploaded:

- [WebRoot]\Download\TaxInvoice\{OrgCode}\{ScheduleID}\

E. SET FOLDER PERMISSIONS:

Set IIS Application Pool Identity permissions on the Download\TaxInvoice folder:

- Read
- Write
- Modify

This is required for the application to:

- Create subfolders automatically
- Upload PDF files

- Download PDF files

VERIFICATION STEPS AFTER DEPLOYMENT:

==========

1. Verify stored procedures exist:

```
SELECT name FROM sys.procedures
WHERE name IN ('GetPaymentDetailsForTaxInvoice', 'SaveSignedTaxInvoiceFile');
```

2. Verify table was created by stored procedure:

```
SELECT name FROM sys.tables WHERE name = 'CourseWisePaymentInvoiceStatus';
```

3. Verify menu item exists:

```
SELECT * FROM MenuMaster WHERE FormName LIKE '%TaxInvoiceUpload%';
```

4. Verify role permissions (CRITICAL):

```
SELECT RoleName, FormName, Enter_Date
FROM RoleForm
WHERE FormName = '~/Forms/DetailsForms/TaxInvoiceUpload.aspx'
ORDER BY RoleName;

-- Expected: At least 3 roles (PAYEE, ADMINISTRATOR/Admin, FINANCE/Finance)
```

5. Check role names in your system:

```
SELECT DISTINCT RoleName FROM RoleForm ORDER BY RoleName;
```

6. Verify folder permissions:

- Navigate to [WebRoot]\Download\TaxInvoice
- Check IIS Application Pool Identity has Read/Write/Modify permissions

7. RESTART IIS (CRITICAL):

- Open PowerShell as Administrator
- Run: iisreset
- This is REQUIRED for database changes to take effect

8. Test the page with ADMIN/FINANCE role:

- Logout and login with ADMINISTRATOR or Finance role
- Navigate to Payments > Tax Invoice Upload
- Select a course schedule
- Verify you see BOTH upload section AND download button
- Try uploading a PDF invoice
- Verify file is saved in correct folder structure

9. Test the page with PAYEE role:

- Logout and login with PAYEE role
- Navigate to Payments > Tax Invoice Upload (or access directly via URL)
- Select appropriate course schedule
- Verify you see ONLY download section (NO upload controls)
- Verify you can download previously uploaded invoices

10. Run final verification script:

File: Database_Scripts\Final_Verification_Check.sql

This will give you a complete status report

TROUBLESHOOTING:

==========

ISSUE 1: PAYEE Users Cannot See the Page

SYMPTOMS:

- PAYEE user logs in but cannot access TaxInvoiceUpload page

- Page redirects to Home.aspx

- Menu item does not appear

DIAGNOSIS:

Run: Database_Scripts\Complete_Diagnostic_PAYEE_Access.sql

This will show you exactly what's missing.

SOLUTIONS:

Solution A - Quick Fix (Most Common):

1. Run: Database_Scripts\QUICK_FIX_All_Roles.sql

   (This adds all possible role name variations)

2. Run: iisreset

3. Clear browser cache (Ctrl+Shift+Delete)

4. Logout and login again

Solution B - Check Role Name:

The issue is usually that the exact role name doesn't match.

1. Query: SELECT DISTINCT RoleName FROM RoleForm WHERE RoleName LIKE '%PAYEE%'

2. Check what it returns (might be 'Payee Organisation', 'PayeeOrganisation', etc.)

3. Add that exact role name to RoleForm table

Solution C - Verify ASP.NET Roles:

1. Query: SELECT * FROM aspnet_Roles WHERE LoweredRoleName LIKE '%payee%'

2. Use the EXACT RoleName from this table

Solution D - Check User's Actual Role:

1. Query: SELECT r.RoleName FROM aspnet_Users u

        INNER JOIN aspnet_UsersInRoles ur ON u.UserId = ur.UserId

        INNER JOIN aspnet_Roles r ON ur.RoleId = r.RoleId

WHERE u.UserName = 'payee_username'

2. Ensure this role has permission in RoleForm table

ISSUE 2: Menu Item Not Showing

SYMPTOMS:

- Page accessible via direct URL

- But menu item doesn't appear in navigation

SOLUTIONS:

1. Verify MenuMaster entry:

```
SELECT * FROM MenuMaster WHERE FormName =
'~/Forms/DetailsForms/TaxInvoiceUpload.aspx';
```

2. Check parent menu item exists:

```
SELECT * FROM MenuMaster WHERE MenuItemCode = 38;
```

3. Refresh view and procedure:

```
EXEC sp_refreshview 'vw_formname';
EXEC sp_recompile 'GetMenuData';
```

4. Restart IIS:

```
iisreset
```

5. Clear browser cache and re-login

ISSUE 3: Upload/Download Rights Not Working Correctly

SYMPTOMS:

- PAYEE sees upload section (should only see download)

- ADMIN doesn't see upload section (should see both)

DIAGNOSIS:

Check code-behind file has correct role detection logic.

SOLUTION:

Verify in TaxInvoiceUpload.aspx.cs:

- Line ~47-48: Both isAdminRole and isPayeeRole flags exist

- Line ~60-62: isAdminRole checks for "ADMIN" in role name

- grdPaymentDetails_RowDataBound method:

  - pnlUploadSection.Visible = isAdminRole (line ~139)

  - pnlDownloadSection.Visible = isAdminRole || isPayeeRole (line ~144)

ISSUE 4: Table Creation Failed

SYMPTOMS:

- Error: "Invalid object name 'CourseWisePaymentInvoiceStatus'"

SOLUTION:

The stored procedure GetPaymentDetailsForTaxInvoice creates this table automatically.

1. Call the stored procedure once manually:

   EXEC GetPaymentDetailsForTaxInvoice @RoleName='ADMIN', @OrganisationCode=0, @ScheduleID=0;

2. Verify table was created:

   SELECT * FROM CourseWisePaymentInvoiceStatus;

ISSUE 5: File Upload Fails

SYMPTOMS:

- Error: "Access denied" or "Could not save file"

SOLUTIONS:

1. Check folder exists:

   [WebRoot]\Download\TaxInvoice\

2. Check IIS App Pool has permissions:

   - Right-click folder > Properties > Security

   - Add: IIS APPPOOL\[YourAppPoolName]

   - Permissions: Read, Write, Modify

3. Check web.config maxRequestLength:

   <httpRuntime maxRequestLength="10240" executionTimeout="300" />

   (10240 KB = 10 MB)

ISSUE 6: Download Returns Wrong File

This should NOT happen with the current implementation because:

- Upload saves file to deterministic path based on OrgCode, ScheduleID, TransactionID

- Download reads path from database (saved by upload)

- Path is built once during upload, never changed

If it still happens:

1. Check CourseWisePaymentInvoiceStatus table:

   SELECT Transaction_id, InvoiceUploadFile, Upload_Status

   FROM CourseWisePaymentInvoiceStatus

   WHERE Transaction_id = 'problematic_transaction_id';

2. Verify physical file exists at that path

ISSUE 7: IIS Not Restarted

SYMPTOMS:

- Database changes done correctly

- But page still doesn't work

SOLUTION:

Database changes (RoleForm, MenuMaster) are cached by ASP.NET.

MUST restart IIS after any database changes:

PowerShell (as Administrator):

iisreset

OR in IIS Manager:

Application Pools > [YourAppPool] > Recycle

IMPORTANT NOTES:

==========

1. ROLE-BASED ACCESS IMPLEMENTATION:

The page code (TaxInvoiceUpload.aspx.cs) enforces access at multiple levels:

a) Page_Load Redirect:

- Only ADMIN/ADMINISTRATOR/FINANCE and PAYEE roles can access the page

- Other roles are redirected to Home.aspx

b) UI Visibility (RowDataBound):

- Upload section (pnlUploadSection): Visible ONLY for ADMIN roles

- Download section (pnlDownloadSection): Visible for ADMIN and PAYEE roles

c) Server-Side Command Validation (RowCommand):

- UploadInvoice command: Executes ONLY for ADMIN roles

- DownloadInvoice command: Executes for ADMIN and PAYEE roles

Role Detection Logic:

- isAdminRole = role name contains "ADMIN" (case-insensitive)

- isPayeeRole = role name contains "PAYEE" (case-insensitive)

- Method: GetPaymentRoleName() uses Roles.GetRolesForUser()

2. MenuItemCode:

- The SQL script uses MenuItemCode = 99

- Before running, verify this number is not already used

- Query to check: SELECT MAX(MenuItemCode) FROM MenuMaster;

- If 99 is taken, use the next available number and update the script

3. ParentMenuItem:

- The script assumes ParentMenuItem = 38 (Payments menu)

- Verify this is correct for your system

- Query: SELECT * FROM MenuMaster WHERE MenuDescription LIKE '%Payment%';

4. Role Names (CRITICAL):

- Role names are CASE-SENSITIVE in database

- ASP.NET role system may use different casing than database

- The automated script covers common variations:

  * PAYEE, Payee

  * ADMINISTRATOR, Admin

  * FINANCE, Finance

- To find YOUR exact role names:

Query 1: SELECT DISTINCT RoleName FROM RoleForm ORDER BY RoleName;

Query 2: SELECT * FROM aspnet_Roles ORDER BY RoleName;

- If your roles have different names/casing, edit the script accordingly

5. Connection String:

- The code uses connection string named "BCASConnectionString"
- Verify this exists in web.config on live server
- Check: <connectionStrings><add name="BCASConnectionString" ... /></connectionStrings>

6. File Size Limit:

- Maximum upload size: 10 MB per file
- Only PDF files are allowed (.pdf extension)
- Validation is enforced in code-behind
- web.config may need maxRequestLength adjustment:

  <httpRuntime maxRequestLength="10240" executionTimeout="300" />

  (10240 KB = 10 MB)

7. MasterPage:

- Page uses ~/MasterPage.master
- NO CHANGES NEEDED IN MASTERPAGE
- Menu is built automatically from MenuMaster table
- Once MenuMaster entry is added, menu item appears automatically for authorized roles

8. Database Table:

- Table CourseWisePaymentInvoiceStatus is created automatically
- No manual table creation needed
- Stored procedure GetPaymentDetailsForTaxInvoice creates it on first run
- Table stores upload status and file path for each transaction

9. File Storage Structure:

- Base path: ~/Download/TaxInvoice/

- Folder structure: {OrgCode}/{ScheduleID}/
- File naming: TaxInvoice_{OrgCode}_{ScheduleID}_{TransactionID}.pdf
- Example: ~/Download/TaxInvoice/1001/5023/TaxInvoice_1001_5023_TXN123456.pdf
- This deterministic structure ensures:
  * No file overwrites
  * Easy traceability
  * Role-based folder access (future enhancement)

10. IIS RESTART REQUIRED:
- After ANY database changes (RoleForm, MenuMaster), you MUST restart IIS
- ASP.NET caches these tables and won't pick up changes otherwise
- Command: iisreset (run as Administrator in PowerShell)
- This is the #1 reason why changes "don't work"

11. Browser Cache:
- After database/IIS changes, users should clear browser cache
- Logout and login again to refresh session/roles
- Or use Ctrl+Shift+Delete to clear cached data

ROLLBACK INSTRUCTIONS (IF NEEDED):

==========

To remove/rollback this feature:

1. Remove role permissions:

```
DELETE FROM RoleForm WHERE FormName = '~/Forms/DetailsForms/TaxInvoiceUpload.aspx';
```

2. Remove menu item:

```
DELETE FROM MenuMaster WHERE FormName = '~/Forms/DetailsForms/TaxInvoiceUpload.aspx';
-- Or if you know the MenuItemCode:
```

-- DELETE FROM MenuMaster WHERE MenuItemCode = 99;

3. Refresh view and procedure:

EXEC sp_refreshview 'vw_formname';

EXEC sp_recompile 'GetMenuData';

4. Restart IIS:

iisreset

5. Drop stored procedures:

DROP PROCEDURE IF EXISTS [dbo].[SaveSignedTaxInvoiceFile];

DROP PROCEDURE IF EXISTS [dbo].[GetPaymentDetailsForTaxInvoice];

6. Remove files from server:

- Delete: BCASApplication\Forms\DetailsForms\TaxInvoiceUpload.aspx

- Delete: BCASApplication\Forms\DetailsForms\TaxInvoiceUpload.aspx.cs

7. Optionally drop tracking table (CAUTION):

-- Only drop if NO other process depends on this table

-- This will DELETE ALL uploaded invoice records

-- DROP TABLE IF EXISTS [dbo].[CourseWisePaymentInvoiceStatus];

8. Optionally remove uploaded files:

- Delete folder: [WebRoot]\Download\TaxInvoice\

- CAUTION: This deletes all uploaded PDF invoices permanently

DEPLOYMENT CHECKLIST:

==========

Pre-Deployment:

[ ] Backup database before making changes

[ ] Note down current MenuMaster max MenuItemCode

[ ] Verify parent menu item code (default: 38 for Payments)

[ ] Check exact role names in your system

[ ] Review and customize SQL scripts if needed

Deployment Steps:

[ ] Copy TaxInvoiceUpload.aspx to server

[ ] Copy TaxInvoiceUpload.aspx.cs to server

[ ] Run Add_TaxInvoiceUpload_Page_Permissions.sql (or manual SQL commands)

[ ] Verify SQL script completed without errors

[ ] Create folder: [WebRoot]\Download\TaxInvoice\

[ ] Set folder permissions (IIS App Pool: Read/Write/Modify)

[ ] Restart IIS (critical!)

[ ] Clear browser cache

Testing:

[ ] Run Final_Verification_Check.sql

[ ] Login as ADMIN/FINANCE role

[ ] Navigate to Payments > Tax Invoice Upload

[ ] Verify upload section is visible

[ ] Upload a test PDF invoice

[ ] Verify file saved to correct folder

[ ] Download the same invoice

[ ] Logout

[ ] Login as PAYEE role

[ ] Access Tax Invoice Upload page (via menu or direct URL)

[ ] Verify upload section is NOT visible

[ ] Verify download section is visible

[ ] Download a previously uploaded invoice

[ ] Logout

Troubleshooting (if needed):

[ ] Run Complete_Diagnostic_PAYEE_Access.sql

[ ] Check IIS Application Pool identity and folder permissions

[ ] Verify role names match database entries

[ ] Run QUICK_FIX_All_Roles.sql if role name issues persist

[ ] Check web.config for BCASConnectionString

[ ] Review IIS logs for errors

Post-Deployment:

[ ] Document any role name variations discovered

[ ] Backup the working SQL scripts

[ ] Inform users about the new feature

[ ] Monitor for any access or upload issues

QUICK REFERENCE - SQL SCRIPT FILES:

==========

File: Add_TaxInvoiceUpload_Page_Permissions.sql

Purpose: Main deployment script - Adds menu item and role permissions

When to use: Initial deployment, adds all necessary database entries

File: QUICK_FIX_All_Roles.sql

Purpose: Adds permissions for all role name variations

When to use: When PAYEE/ADMIN cannot see page despite main script

File: Complete_Diagnostic_PAYEE_Access.sql

Purpose: Comprehensive diagnostic for PAYEE visibility issues

When to use: Troubleshooting why PAYEE users cannot access page

File: Final_Verification_Check.sql

Purpose: Post-deployment verification

When to use: After deployment to confirm everything is configured correctly

File: Debug_TaxInvoice_Permissions.sql

Purpose: Basic permission and menu diagnostics

When to use: Quick check of current database state

File: Fix_PAYEE_Role_Variations.sql

Purpose: Adds PAYEE role name variations only

When to use: Specifically fixing PAYEE access issues

COMMON QUERIES FOR MAINTENANCE:

==========

```
-- Check which roles have access:
SELECT RoleName, FormName
FROM RoleForm
WHERE FormName = '~/Forms/DetailsForms/TaxInvoiceUpload.aspx'
ORDER BY RoleName;

-- Check menu item configuration:
SELECT MenuItemCode, MenuDescription, ParentMenuItem, FormName, WhetherLeafMenu
FROM MenuMaster
WHERE FormName = '~/Forms/DetailsForms/TaxInvoiceUpload.aspx';

-- Check upload status for a specific schedule:
SELECT Transaction_id, ProformaInvoiceNo, Date_of_Payment,
    Amount_Recieved, InvoiceUploadFile, Upload_Status
```

```
FROM CourseWisePaymentInvoiceStatus
WHERE Schedule_ID = 5023  -- Replace with your schedule ID
ORDER BY Date_of_Payment DESC;

-- Find all uploaded invoices:
SELECT COUNT(*) AS TotalUploaded
FROM CourseWisePaymentInvoiceStatus
WHERE Upload_Status = 'Uploaded';

-- Find pending uploads:
SELECT Transaction_id, ProformaInvoiceNo, Amount_Recieved, Date_of_Payment
FROM CourseWisePaymentInvoiceStatus
WHERE Upload_Status = 'Not Uploaded'
ORDER BY Date_of_Payment DESC;

-- Check all role names in system:
SELECT DISTINCT RoleName FROM RoleForm ORDER BY RoleName;

-- Find user's actual role:
SELECT u.UserName, r.RoleName
FROM aspnet_Users u
INNER JOIN aspnet_UsersInRoles ur ON u.UserId = ur.UserId
INNER JOIN aspnet_Roles r ON ur.RoleId = r.RoleId
WHERE u.UserName = 'your_username';  -- Replace with actual username
```

TECHNICAL IMPLEMENTATION DETAILS:

==========

Code-Behind Logic (TaxInvoiceUpload.aspx.cs):

---

1. Role Detection:

- Method: GetPaymentRoleName()
- Returns: Role-specific value based on Roles.GetRolesForUser()
- Flags: isAdminRole (contains "ADMIN"), isPayeeRole (contains "PAYEE")

2. Access Control Layers:

- Layer 1 (Page_Load): Redirects non-ADMIN/non-PAYEE to Home.aspx
- Layer 2 (RowDataBound): Controls visibility of upload/download panels
- Layer 3 (RowCommand): Validates role before executing upload/download

3. File Upload Flow:

a) Validate: PDF only, max 10MB, transaction belongs to user's org
b) Build paths: BuildInvoiceRelativeFolder(), BuildInvoiceFileName()
c) Save file: Server.MapPath() + FileUpload.SaveAs()
d) Update DB: SaveSignedTaxInvoiceFile() stored procedure
e) Refresh grid: BindData()

4. File Download Flow:

a) Validate: File exists, transaction belongs to user's org/role
b) Read: InvoiceUploadFile column from CourseWisePaymentInvoiceStatus
c) Stream: Response.TransmitFile() with content-disposition header
d) Sanitize: Filename cleaned to prevent path traversal

Database Schema:

Table: CourseWisePaymentInvoiceStatus

- Primary Key: Transaction_id (VARCHAR(50))
- File tracking: InvoiceUploadFile, Upload_Status
- Metadata: OrgCode, ScheduleID, Amount, Date, etc.
- Sync: LastSyncedOn (updated on every data refresh)

Stored Procedures:

- GetPaymentDetailsForTaxInvoice: Creates table, syncs payment data, returns filtered results

- SaveSignedTaxInvoiceFile: Updates InvoiceUploadFile and Upload_Status for a transaction

Security Features:

---

1. Role-based access at page, UI, and command levels

2. Organization code validation (user can only upload for their org)

3. File extension validation (PDF only)

4. File size validation (10MB limit)

5. Path traversal protection (filename sanitization)

6. Deterministic file paths (prevents overwrites and conflicts)

7. Server-side command validation (prevents client-side bypass)

==========

END OF DEPLOYMENT INSTRUCTIONS

==========